HIDRO SMART · ENTREGA DE PROYECTO FORMATIVO

Artículo del proyecto

Documento de trabajo para revisión del equipo

Borrador de revisión · no es versión aprobada

Corte documental: 2026-11-04
Fuente funcional acordada: PT_HS · hu-20-dev · 1f02b78
Carpeta: 09-project-article

> **Estado y alcance.** Documento en revisión para la entrega. Las fechas, cifras y funciones heredadas del documento fuente solo se consideran vigentes cuando están respaldadas por evidencia del monorepo PT_HS. Consulta el índice de entrega y el estado del borrador.

HIDROSMART

HidroSmart: plataforma modular para monitoreo inteligente del consumo de agua

Artículo técnico del proyecto

Versión documental consolidada 1.0
Fecha de corte: 2026-10-09
Código documental: DOC-12

Resumen

HidroSmart es una plataforma orientada a servicios que integra sensores ESP32, MQTT, una API Express, PostgreSQL y clientes web/móvil para observar el consumo de agua. El sistema separa la captura, el transporte, la ingestión, la persistencia y la visualización. En el corte documentado se verificó la ejecución local, la cadena de ingestión y una prueba física de telemetría; permanecen pendientes la seguridad MQTT de producción, la calibración con volumen conocido, el backup/restore y pruebas formales de accesibilidad y experiencia móvil.

Palabras clave: consumo de agua; IoT; MQTT; ESP32; PostgreSQL; React; telemetría.

1. Introducción

La gestión del agua requiere información oportuna y comprensible. Una lectura aislada no es suficiente: debe relacionarse con hogares, dispositivos, periodos, metas y alertas. HidroSmart aborda este problema mediante un sistema que conecta medición física y software de consulta, conservando una ruta trazable desde el sensor hasta la interfaz.

2. Propuesta tecnológica

El firmware para ESP32 obtiene pulsos y flujo, guarda parámetros de red por BLE y publica telemetría en MQTT. El backend suscribe, valida, normaliza, deduplica y persiste la lectura. PostgreSQL organiza datos por dominios y Liquibase controla su evolución. React/Vite presenta la información en web; la experiencia móvil utiliza WebView y un puente BLE para configuración.

3. Metodología

Revisión de requisitos y documentos del proyecto.

Inspección de la estructura del repositorio y sus ramas.

Contraste entre documentación histórica y fuente de código.

Identificación de evidencia de ejecución local, pruebas de backend, compilación y hardware.

Clasificación de resultados en verificado, histórico o pendiente.

4. Resultados documentados

Indicador	Resultado del corte 2026-09-26
Base de datos	245 changesets, 58 tablas de aplicación y 5 vistas materializadas reportadas.
Backend	174 pruebas unitarias aprobadas reportadas.

Frontend	Lint, typecheck y build aprobados; aplicación respondía en localhost:5173.
Firmware	Compilación reportada; 17.6% RAM y 82.0% Flash.
Hardware	5 lecturas en flujo y 6 en reposo persistidas para HS-141F47470968.
Estado observado	ONLINE/MQTT_CONNECTED, 12.332 L/min, 463 pulsos, 1.029 L por intervalo y RSSI -48 dBm en la evidencia citada.

5. Discusión

La separación de responsabilidades facilita la evolución: el broker no necesita conocer la presentación y la UI no accede directamente a la base. El contrato de telemetría y la deduplicación reducen inconsistencias en reintentos. Sin embargo, un prototipo local no demuestra por sí mismo seguridad, rendimiento, recuperación ni exactitud metrológica en producción. También debe distinguirse el flujo MQTT de la actualización visual, que en el corte usa polling HTTP.

6. Limitaciones y trabajo futuro

TLS, autenticación y ACL del broker para producción.

Calibración con volumen patrón y análisis de error.

Prueba automatizada de backup y restauración.

Entrega integral de correos transaccionales (Brevo) y conexión de disparadores de notificación.

E2E, accesibilidad y validación en dispositivos móviles.

Definición de SLA, pruebas de carga y observabilidad de producción.

Confirmación institucional del código de proyecto, ficha y periodo.

7. Conclusiones

HidroSmart cuenta con una base técnica coherente para monitoreo de consumo: dispositivos, mensajería, API, persistencia y clientes se encuentran organizados en un repositorio integrado. La evidencia disponible demuestra avances funcionales y una prueba física inicial. La conclusión responsable es de preparación para cierre documental y validación adicional, no de producción plena, hasta cerrar las acciones de seguridad, calibración, recuperación y pruebas de aceptación.

Referencias

Resultados del artículo: PT_HS, README.md y docker-compose.yml, rama hu-11-dev, commit b54732e (corte histórico). Estado posterior del código: PT_HS/hu-20-dev, commit `1f02b78`; los resultados experimentales no se recalcularon.

PT_HS/docs/estado-verificado-2026-09-26.md.

PT_HS/BD_HS/docs/README.md; PT_HS/BK_HS/docs/00-estado-actual.md; PT_HS/BK_HS/docs/14-mqtt-protocol.md.

PT_HS/firmware/README.md y platformio.ini.

Reporte Proyecto Formativo – 2678382 – DESARROLLO DE SOFTWARE ORIENTADO A SERVICIOS, PDF local revisado.

Nota de control documental. El artículo usa únicamente resultados y cifras presentes en la evidencia revisada. No atribuye métricas de rendimiento, exactitud o impacto de ahorro que no estén medidas.

Nota de actualización - 2026-11-04

Los resultados y métricas de este artículo corresponden al corte histórico de 2026-09/10 y no se recalcularon. En PT_HS hu-20-dev (`1f02b78`) el código conserva el envío transaccional con Brevo por API HTTPS y el perfil MQTT/TLS de Flespi. La

respuesta aceptada por un proveedor no prueba la entrega de todos los correos; la configuración de ACL, la calibración y las pruebas físicas requieren evidencia del entorno. Esta nota no modifica los resultados experimentales del artículo. Consulta el estado del borrador.